

Ravan Muradzada

1) `[root@mhd1 ~]# useradd mr-ravan`

It has created a new user named `mr-ravan` here.

```
[root@mhd1 ~]# passwd mr-ravan
Changing password for user mr-ravan.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
```

I have set password here for new user.

3) `[root@mhd1 ~]# su - mr-ravan`

It switches to another user.

4) `[mr-ravan@mhd1 ~]$ whoami`
`mr-ravan`

`whoami` gives our username. We have switched to another user, so our username is changed, too.

```
[mr-ravan@mhd1 ~]$ ls -a > newfile11.log
[mr-ravan@mhd1 ~]$ cat newfile11.log
.
..
.bash_logout
.bash_profile
.bashrc
newfile11.log
```

`ls -a` shows all files including hidden ones. `>` redirects the output to specified file.

```
[mr-ravan@mhd1 sysconfig]$ touch newFile.txt 2>> ~/newfile11.log
[mr-ravan@mhd1 sysconfig]$ cat ~/newfile11.log
.
..
.bash_logout
.bash_profile
.bashrc
newfile11.log
6) touch: cannot touch 'newFile.txt': Permission denied
```

I used `2>>`, because I want to append (not overwrite) std err to specified file. In this directory for normal users, creating a new file is denied, thus error occurred and I have redirected this error to our log file.

```
[mr-ravan@mhd1 ~]$ touch ~/olanfayl /etc/sysconfig/yenifile &>> file_creation.log
[mr-ravan@mhd1 ~]$ cat file_creation.log
7) touch: cannot touch '/etc/sysconfig/yenifile': Permission denied
```

`&>>` helps us to redirect both std out and std err to the same file. (It is appending, not overwriting).

```
[mr-ravan@mhd1 ~]$ ls /home /nonexistent_dir > output.log 2> error.log
[mr-ravan@mhd1 ~]$ cat output.log
/home:
idtech-mrravan
mr-ravan
[mr-ravan@mhd1 ~]$ cat error.log
8) ls: cannot access '/nonexistent_dir': No such file or directory
```

```
9) touch /etc/sysconfig/file.txt 2> /dev/null
```

10)

- a. > - Overwrites std output.
- b. >> - Appends std output without overwriting.
- c. 1> - Same with >
- d. 1>> - Same with >>
- e. 2> - Overwrites std err.
- f. 2>> - Appends std err without overwriting.
- g. > 2>&1 (or just &>) - Overwrites both std out and std err to the same file.
- h. >> 2>&1 (or just &>>) - Appends both std out and std err to the same file.
- i. > /dev/null - Sends any std out result to null, discards and does not save anywhere.
- j. 2> 1.log 1> 2.log - This overwrites std out to 2.log, std.err to 1.log.
- k. | (pipe) – It takes the output of the first command and gives it to the second command as an input

11)

- 1. \$ is actually a special character and used to represent variables itself. Its usage inside string can confuse the system. We should use single quotes, because single quotes think everything inside as a string.
- 2. Using `!` multiple times (like `!!`, `!!!`) can trigger history expansion, causing Bash to try replacing it with previous commands, which may lead to unexpected results or errors.
- 3. “ ” (double quotes) - Allow variable expansion and command substitution.
‘ ’ (single quotes) - Everything inside them is taken literally. No variable expansion, no command substitution, no special character meaning.
`` (backlist) – Used for command substitution. They execute a command and replace it with its output. (Example: `echo `pwd``)

4. Double quotes can contain variable expansions like if we have a variable of `x=5`, we can express it in a string like `echo "My variable is $x"`. The result will be `"My variable is 5"`.

However, for single quote there is no variable expansion. Even if we have a variable of `x=5` and we write `echo 'My variable is $x'`, the result will be just `"My variable is $x"`.

5. It is used to represent an output of any command. Example: `echo `ls``. As a result, it will run `ls` command. Moreover, it can be used inside files which contain runnable commands. If other quotes are used instead, the command will not be run.